



22. Swapping Policies

Goal of Cache Management

Goal

Metric

The Optimal Replacement Policy

Tracing the Optimal Policy

A Simple Policy: FIFO

Advantages

Disadvantages

Tracing the FIFO policy

BELADY's ANOMALY

Another Simple Policy: Random

Random Performance

Using History

Using History: LRU

Workload Examples

The No-Locality Workload

Workload Example : The 80-20 Workload

Workload Example : The Looping Sequential

+) 왜 LRU와 FIFO가 최악인지 쉽게 설명

Approximating LRU: Clock Algorithm

Purpose

Use Bit Mechanism

Clock Algorithm Steps

+) 추가 설명

하드웨어가 하는 일 (참조 시점)

OS가 하는 일 (교체 시점)

+) 예시

Goal of Cache Management

Goal

Minimize the number of cache misses

Metric

Reduce the Average Memory Access Time(AMAT)



$$AMAT = (P_{HIT} * T_M) + (P_{MISS} * T_P)$$

T_M	The cost of accessing memory
T_P	The cost of accessing disk
P_Hit	The probability of finding the data item in the cache (a hit)
P_Miss	The probability of not finding the data in the cache (a miss)

The Optimal Replacement Policy

- 전체적으로 가장 적은 miss를 발생시키는 정책
 - 미래에 가장 늦게 접근될 페이지를 교체
 - 가능한 한 가장 적은 캐시 miss가 발생
- 이걸 실제로 구현할 수 없어. 왜냐하면 미래를 알아야 해서...
 - 오직 비교 기준점(comparison point)으로만 사용, 다른 알고리즘(LRU, FIFO, Clock 등)이 완벽한 성능에 얼마나 가까운지 측정하는 용도

Tracing the Optimal Policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit or Miss	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2

1	Hit		0, 1, 2
3	Miss	2	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

Hit Rate is Hits / (Hits + Misses) = 54.6%

A Simple Policy: FIFO

- Pages are queued in the order they enter the system.
- When a replacement occurs, the page at the tail of the queue (the oldest one = the "First-in" pages) is evicted.

Advantages

- 구현하기에 간단하고 쉬움

Disadvantages

- page의 중요성이나 usage frequency를 고려하지 않음

Tracing the FIFO policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit or Miss	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	0	1, 2, 3
0	Miss	1	2, 3, 0
3	Hit		2, 3, 0

1	Miss		3, 0, 1
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

Hit Rate is Hits / (Hits + Misses) = 36.4%

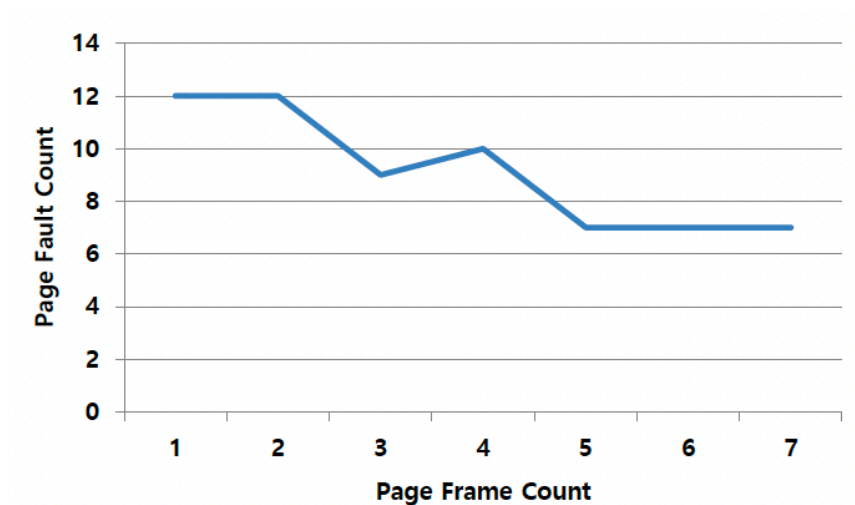
page 0이 겁나 많이 조회되었는데도, FIFO가 still kicks it out

BELADY'S ANOMALY

- 일반적으로 cache 크기가 커지면 hit rate 또한 개선되어야함
- FIFO 정책에서는 캐시 크기가 커졌는데 오히려 hit rate가 떨어지는 현상이 발생할 수 있음
 - FIFO는 사용 패턴을 전혀 고려하지 않고 그냥 먼저 들어온 순서대로 쫓아내기 때문

Reference Row

1 2 3 4 1 2 5 1 2 3 4 5



- 캐시 크기가 3 → 4로 갈 때 page fault가 늘어나는 모습 확인 가능

Another Simple Policy: Random

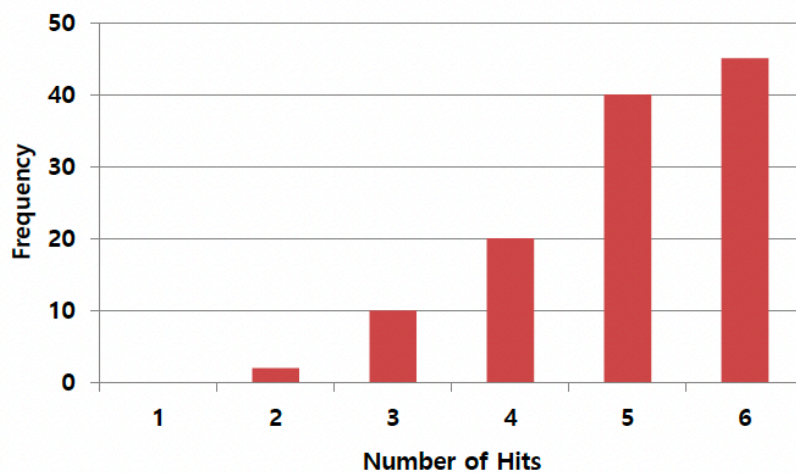
- 메모리 압박이 있을 때 랜덤하게 쫓아낼 페이지를 선택
- 어떤 페이지가 중요한지, 자주 사용되는지 전혀 고려하지 않아
- 랜덤 선택이 얼마나 운 좋게 맞아떨어지느냐에 따라 성능이 달라짐
 - 구현이 간단하고 빠름

- 그러나, 운이 나쁘면 성능이 **형편 없을 수 있다**

Access	Hit or Miss	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	0	1, 2, 3
0	Miss	1	2, 3, 0
3	Hit		2, 3, 0
1	Miss	3	2, 0, 1
2	Hit		2, 0, 1
1	Hit		2, 0, 1

Random Performance

- Sometimes, Random is as good as optimal, achieving 6 hits on the example trace



Random Performance over 10,000 Trials

Using History

- Learn on the past and use history.

- Two type of historical information

Historical Information	Meaning	Algorithms
recency (최근성)	최근에 접근된 페이지일수록 다시 접근될 가능성이 높다	LRU
frequency (빈도)	많이 접근된 페이지는 분명히 가치가 있으니 교체하면 안 된다	LFU

Using History: LRU

- Replace the least-recently-used page

Reference Row

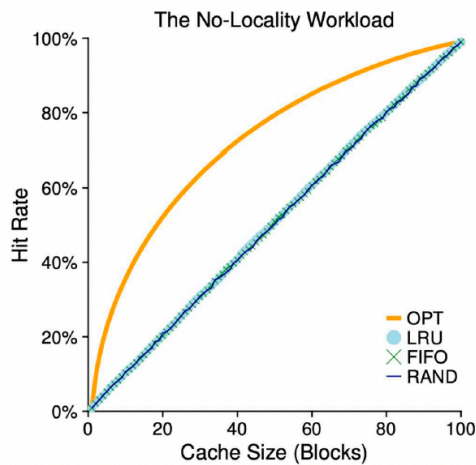
0 1 2 0 1 3 0 3 1 2 1

Access	Hit or Miss	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		1, 2, 0
1	Hit		2, 0, 1
3	Miss	2	0, 1, 3
0	Hit		1, 3, 0
3	Hit		1, 0, 3
1	Hit		0, 3, 1
2	Miss	0	3, 1, 2
1	Hit		3, 2, 1

Workload Examples

The No-Locality Workload

- **지역성이 전혀 없는 workload**
 - 각 페이지 참조가 접근 가능한 페이지들 중에서 **완전히 랜덤하게** 선택
 - 페이지들이 자주 재사용되거나 예측 가능한 패턴으로 접근되지 않는다



- OPT = 이론적으로 달성 가능한 최고의 hit rate
- LRU, FIFO, RAND는 거의 동일한 성능
왜냐하면 미래 접근들이 random이어서 어떤 정책도 재사용을 효과적으로 예측할 수 없어서
- cache 크기가 작을 때는 모든 정책에서 hit rate가 작음
- cache 크기가 커질수록 hit rate가 점진적으로 개선되면서 OPT에 가까워짐

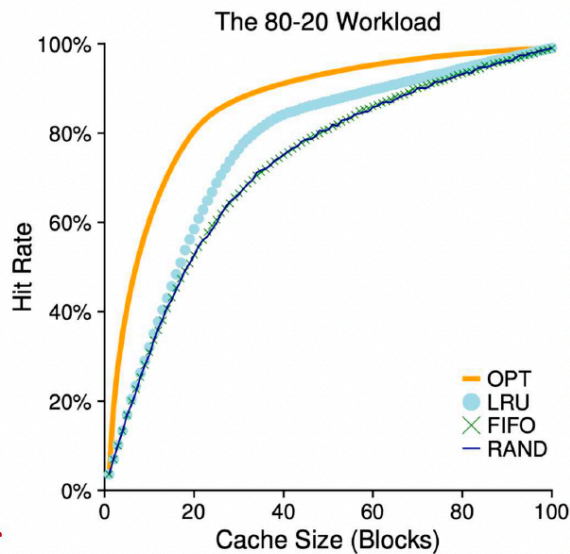


결론: No-locality 워크로드에서는 LRU, FIFO, Random 같은 교체 정책들이 비슷한 성능을 보임

왜? 시간적 지역성이나 공간적 지역성이 없기 때문에

Workload Example : The 80-20 Workload

- 지역성이 있는 workload
 - 전체 메모리 참조의 80%가 전체 페이지 중 20%에만 집중 - 이 20%를 hot pages라고 함
 - 나머지 20%의 참조는 나머지 80%의 페이지들에 분산
 - 실제 현실 세계의 workload를 반영



- OPT = 이론적으로 달성 가능한 최고의 hit rate
- LRU는 FIFO나 RAND보다 **OPT에 더 가까운 성능**
→ LRU는 최근에 사용된 (hot) 페이지들을 메모리에 유지하는 경향이 있어서
- FIFO와 Random 정책은 hot 페이지들을 너무 일찍 쫓아낼 수 있어서, LRU에 비해 히트율이 낮아짐

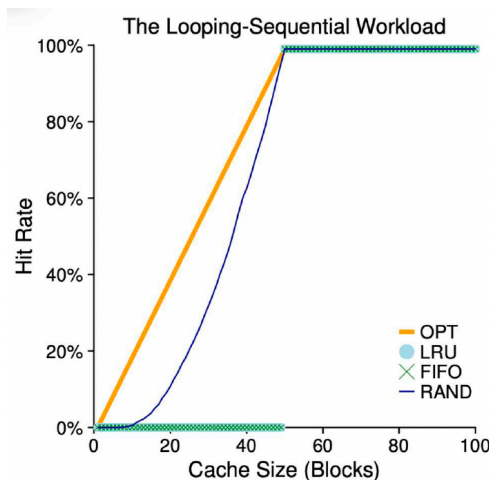


지역성이 풍부한 워크로드에서는 **LRU가 FIFO와 RAND를 압도**

왜? LRU가 **시간적 지역성(temporal locality)에 적응** → 자주 사용되는 페이지들을 캐시에 더 오래 유지

Workload Example : The Looping Sequential

- Workload 특성
 - 프로그램이 **50개의 페이지를 순차적으로** 접근
 - 페이지 0에서 시작해서 1, 2, ... 49까지 갔다가 다시 처음으로 돌아와서 반복
 - 이 접근 패턴은 **강한 규칙성(strong regularity)**을 가지고 있음.
그러나, **시퀀스가 한 바퀴 돌고 나면 시간적 중첩(temporal overlap)이 없어.**
 - 시간적 중첩이란?: 같은 페이지를 짧은 시간 안에 다시 쓰는 것
 - 같은 페이지를 다시 접근하기까지 **너무 오래 걸린다** = 그 사이에 캐시에서 쫓겨난다 = **LRU가 소용없다**



- 캐시가 50개 페이지를 전부 담을 수 있으면 OPT는 완벽한 히트율을 달성
- **LRU와 FIFO**는 캐시 크기가 루프 길이(50 페이지)보다 작을 때 **매우 형편없는 성능**
→ 루프가 다시 시작할 때쯤이면 이전에 사용했던 페이지들이 전부 쫓겨나 있어서
- 흥미롭게도, **RAND**는 캐시 크기가 루프 길이보다 작을 때 가끔 OPT에 근접한 성능을 낼 수 있음
→ 랜덤한 선택이 우연히 접근 패턴과 유리하게 맞아떨어질 때가 있기 때문

+) 왜 LRU와 FIFO가 최악인지 쉽게 설명

- 예를 들어 캐시 크기가 49, 페이지 0, 1, 2, ... 48까지 캐시에 들어가 있음
- 이제 페이지 49를 넣어야 하는데
 - LRU는 "가장 오래 전에 쓴 거"인 **페이지 0**을 쫓아내. 그 다음 다시 루프 시작하면? 페이지 0이 필요한데 방금 쫓아냈으니 **miss!** 그리고 페이지 0 넣으려면 이번엔 페이지 1을 쫓아내고... 이게 계속 반복
 - FIFO도 마찬가지로 최악의 성능을 보임

Approximating LRU: Clock Algorithm

Purpose

- **LRU(Least Recently Used) page replacement**를 근사(approximation)하는 페이지 교체 알고리즘
- 목적: **use bit**라는 간단한 하드웨어 지원 메커니즘을 사용해서 **overhead 줄이기**

Use Bit	Meaning
0	Evict the page (이 페이지를 쫓아냄)
1	Clear the Use bit and advance the hand (Use bit를 0으로 clear하고 다음 페이지로 이동)

Use Bit Mechanism

- 각 페이지마다 **use bit** (reference bit라고도 불림)가 있음
- 페이지가 참조되면 **하드웨어가 자동으로** use bit를 1로 설정
- 하드웨어는 절대 이 비트를 리셋하지 않음. 비트를 0으로 클리어하는 건 **운영체제의 책임(system's responsibility)**

Clock Algorithm Steps

1. 모든 페이지가 **원형 리스트**로 배열돼 있어. 시계처럼 생겨서 clock
2. 시계 바늘(clock hand)이 한 번에 하나의 페이지를 가리켜.
3. 페이지 교체가 필요할 때:
 - a. use bit = 1 ⇒ 비트를 0으로 클리어하고 바늘을 다음 페이지로 이동
"한 번 기회를 줘": 0으로 바꾸고 넘어가기
 - b. use bit = 0 ⇒ 이 페이지는 최근에 안 쓰인 걸로 간주하고 **쫓아냄**
"기회 줬는데도 안 쓰였네? 쫓아냄"
4. use bit = 0인 페이지를 찾을 때까지 이 과정을 반복

+) 추가 설명

하드웨어가 하는 일 (참조 시점)

- 프로그램이 어떤 페이지를 읽거나 쓰면, 하드웨어가 **자동으로** 그 페이지의 use bit를 1로 설정
- 시계 바늘이랑 상관없이 **항상** 일어남

OS가 하는 일 (교체 시점)

- 페이지 교체가 필요할 때만 OS가 시계 바늘을 돌리면서 use bit를 확인하고 0으로 클리어

+) 예시

각 페이지는 use bit를 가지고 있고, 모두 1인 상태이고 새 페이지 E를 넣어야함

A(1) - B(1) - C(1) - D(1)
↑
바늘

1단계: 바늘이 A를 봐. use bit = 1이네?

- "아 최근에 쓰였구나, 봐줄게. 근데 다음엔 모른다?"
- use bit를 0으로 바꾸고 다음으로 이동

A(0) - B(1) - C(1) - D(1)
↑
바늘

2단계: 바늘이 B를 봐. use bit = 1이네?

- 똑같이 0으로 바꾸고 다음으로 이동

3단계: 바늘이 C를 봐. use bit = 1이네?

- 0으로 바꾸고 다음으로 이동

4단계: 바늘이 D를 봐. use bit = 1이네?

- 0으로 바꾸고 다음으로 이동 (한 바퀴 돌아서 다시 A로)

A(0) - B(0) - C(0) - D(0)
↑
바늘

- "너 아까 한 번 봐줬는데 그 사이에도 안 쓰였네? 나가!"
→ **A를 쫓아내고 E를 넣음**